

NRF24L01

The walkie-talkie of IoT: A lightweight alternative to WiFi/Bluetooth for embedded systems

Any IoT devices can communicate wirelessly using NRF24L01+ modules, as long as they share the same channel and address.

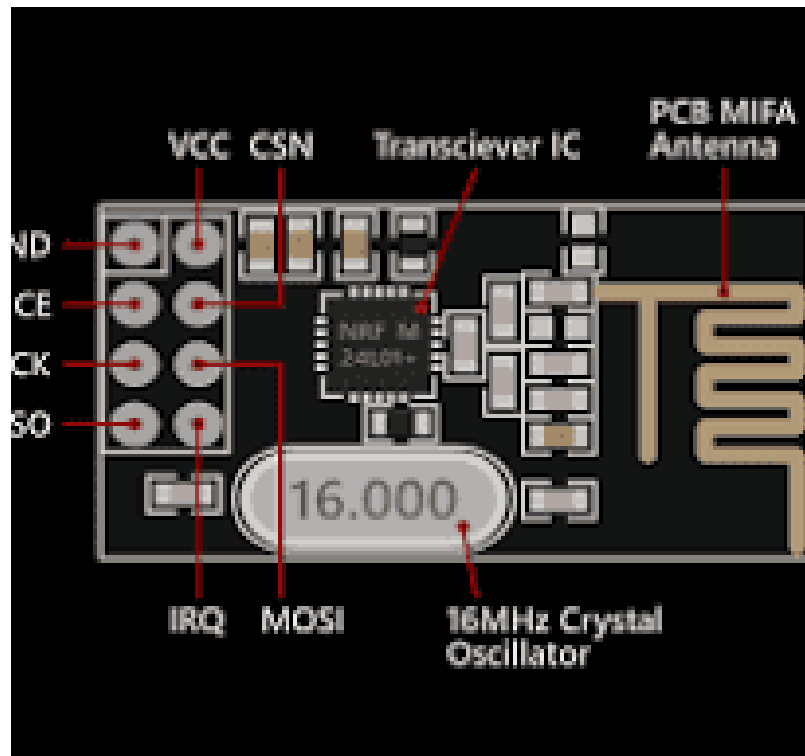


NRF24L01+ Hardware Overview

NRF24L01+ Module

- Small, cheap RF module (~\$1–\$2)
- On-board antenna (or external SMA)
- Communicates with MCU via **SPI**
- Range: up to **100m** (line of sight)
- Data rate: 250 Kbps / 1 Mbps / 2 Mbps

NRF24L01+ Pin Layout



Pin	Name	Description
1	GND	Ground
2	VCC	3.3V power (NOT 5V!)
3	CE	Chip Enable (TX/RX mode)
4	CSN	Chip Select (SPI)
5	SCK	SPI Clock
6	MOSI	SPI Data (MCU → Module)
7	MISO	SPI Data (Module → MCU)
8	IRQ	Interrupt (optional)

⚠ 3.3V only — 5V will damage the module!

How SPI Works with NRF24L01+

SCK	clock signal
MOSI	data to NRF24L01+
MISO	data from NRF24L01+
CSN	select this device
CE	activate TX or RX mode

- SPI is a **synchronous serial bus**
- Master (MCU) controls the clock
- CE pin switches between **transmit** and **receive**

Addressing: Pipes

NRF24L01+ uses "**pipes**" to address communication

- Up to **6 receive pipes** on one module
- Each pipe has a unique **5-byte address**
- Like mailbox numbers on the same street

```
Module A writes to address: 0xF0F0F0F0E1  
Module B listens on:       0xF0F0F0F0E1
```

Both modules must agree on the **same address**.

Arduino code

Setting up a simple transmitter and receiver using the RF24 library:

```
const byte address[6] = "Node1"; // 5 chars + null terminator

// Transmitter
radio.openWritingPipe(address);
radio.stopListening();

// Receiver
radio.openReadingPipe(1, address);
radio.startListening();
```

Multiple Pipes

```
// Hub (Receiver) – listening on 6 pipes simultaneously
const byte addresses[6][6] = {
    "Sens1", "Sens2", "Sens3",
    "Sens4", "Sens5", "Sens6"
};

for (int i = 0; i < 6; i++) {
    radio.openReadingPipe(i, addresses[i]);
}
radio.startListening();
```

[Sensor 1] → pipe 0 → "Sens1"
[Sensor 2] → pipe 1 → "Sens2" → [Hub]
[Sensor 3] → pipe 2 → "Sens3"

Channels

NRF24L01+ supports **125 channels** (0–124)

- Channel = specific frequency offset
- Channel 76 → 2.476 GHz
- Avoid channels 0–2 (used by WiFi channel 1)

Both transmitter and receiver must use **the same channel**.

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

RF24 radio(7, 8); // CE=7, CSN=8
const byte address[6] = "00001";

void setup() {
    radio.begin(); // ← sets channel 76 by default
    // radio.setChannel(100); // ← explicit is better practice
    radio.openWritingPipe(address);
    radio.setPALevel(RF24_PA_MIN);
    radio.stopListening();
}

void loop() {
    const char text[] = "Hello Arduino";
    radio.write(&text, sizeof(text));
    delay(1000);
}
```

Arduino Example: Receiver

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

RF24 radio(7, 8);
const byte address[6] = "00001";

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.openReadingPipe(0, address);
  radio.setPALevel(RF24_PA_MIN);
  radio.startListening();
}

void loop() {
  if (radio.available()) {
    char text[32] = "";
    radio.read(&text, sizeof(text));
    Serial.println(text);
  }
}
```

Transmitter vs Receiver Flow

Transmitter

```
openWritingPipe(addr)  
stopListening()  
radio.write(data)
```

↔

Receiver

```
openReadingPipe(1, addr)  
startListening()  
radio.available()  
radio.read(32)
```

Key idea: **One writes, the other listens**

The address connects them.

The Big Question: What Do You Actually Send?

NRF24L01+ gives you a raw radio link.

You can send **up to 32 bytes** per packet.

But **what do those 32 bytes mean?**

```
Sender sends:  [ 23, 01, 00, 72, 00, ... ]  
Receiver gets: [ 23, 01, 00, 72, 00, ... ]
```

The radio delivers the bytes.

You decide what they mean.

You Must Define Your Own Protocol

NRF24L01+ has **no built-in protocol** — just raw bytes.

You need to decide:

- What is byte 0? (sensor type? command?)
- What is byte 1–2? (a temperature value?)
- Is it big-endian or little-endian?
- How does the receiver know the data is valid?

This is called designing an **application-level protocol**.

Example: DIY Packet Format

Let's say we want to send a temperature reading:

Byte 0	:	Sensor ID	(e.g., 0x01 = Sensor 1)
Byte 1	:	Data type	(e.g., 0xA1 = temperature)
Byte 2–3	:	Value	(e.g., 0x00F0 = 24.0°C)
Byte 4	:	Checksum	(simple XOR of bytes 0–3)

Both sender and receiver must follow this **exact same layout**.

Arduino Code: Sending a Struct

A clean way: use a C `struct` as your packet format.

```
struct Packet {  
    uint8_t  sensorId;    // 1 byte  
    uint8_t  dataType;    // 1 byte  
    int16_t  value;       // 2 bytes (e.g., temp * 10)  
    uint8_t  checksum;    // 1 byte  
};  
  
Packet p;  
p.sensorId = 1;  
p.dataType = 0xA1;  
p.value     = 240; // 24.0°C  
p.checksum  = p.sensorId ^ p.dataType ^ (p.value & 0xFF);
```

Arduino Code: Receiving the Struct

```
struct Packet {
    uint8_t  sensorId;
    uint8_t  dataType;
    int16_t  value;
    uint8_t  checksum;
};

void loop() {
    if (radio.available()) {
        Packet p;
        radio.read(&p, sizeof(p));

        // Validate checksum
        uint8_t expected = p.sensorId ^ p.dataType ^ (p.value & 0xFF);
        if (p.checksum == expected) {
            Serial.print("Temp: ");
            Serial.println(p.value / 10.0);
        }
    }
}
```

Comparison with Bluetooth

Revisit: Bluetooth Profiles

With NRF24L01+, you invented everything above yourself.

With **Bluetooth**, this problem is already solved by **profiles**.

A **Bluetooth Profile** is a standardized specification that defines **what data is sent and how it is formatted**.

No guessing. No custom packet design. Just use the profile.

A profile answers:

- **What service** does this device provide?
- **What data format** is used?
- **How does the receiver interpret** each byte?

Examples:

Profile	Purpose	Data Format
HRM (Heart Rate)	Send heart rate	BPM as standard byte
HID	Keyboard/Mouse	Key codes, movement
Serial (SPP)	Generic serial data	Free-form bytes
BLE GATT	Sensors, IoT	Structured characteristics

Bluetooth Profile in Action

A heart rate monitor (HRM) uses the **Heart Rate Profile**:

Bluetooth HRM packet (standardized):

Byte 0: Flags (0x00 = 8-bit HR value)

Byte 1: Heart Rate Value (e.g., 72 BPM)

Any app on any phone that supports HRM profile can read it — iPhone, Android, smartwatch, laptop — **all speak the same language.**

With NRF24L01+, only your own code on your own MCU can decode it.

Side-by-Side Comparison

NRF24L01+

You define packet format
You write encode/decode
Only your code reads it
Fast to prototype

vs.
vs.
vs.
vs.

Bluetooth (BLE)

Profile defines it
Library handles it
Any standard app reads it
More setup, but portable

NRF24L01+: Full control, but full responsibility

Bluetooth: Less control, but works everywhere

When Do You Need a Protocol?

Situation	Recommendation
Two Arduinos talking to each other	Simple struct is enough
Pi collecting from 6 sensors	Define a clear packet format
Phone app reading sensor data	Use Bluetooth (BLE + GATT)
Wearable → phone	Bluetooth HRM/HID profile
Industrial sensor to server	Use MQTT over WiFi

The right tool depends on who needs to understand the data.

Comparison: NRF24L01+ vs Bluetooth

Feature	NRF24L01+	Bluetooth
Frequency	2.4 GHz	2.4 GHz
Standard	Proprietary	Bluetooth SIG
Smartphone Compatible	✗ No	✓ Yes
Built-in Protocol Stack	✗ No	✓ Yes

Same frequency, but different communication system

Use Cases: NRF24L01+ vs Bluetooth

Use Case	NRF24L01+	Bluetooth
MCU ↔ MCU	✓ Excellent	Possible
Phone ↔ MCU	✗ No	✓ Yes
Low Latency	✓ Very Low	Moderate
Power Efficiency	✓ Very High	BLE = High

- Bluetooth: Like a structured phone network (with rules, IDs, and services)
- NRF24L01+: Like a walkie-talkie (set the channel and talk)

NRF24L01+ vs ESP8266/ESP32 WiFi

Feature	NRF24L01+	ESP8266/ESP32
Protocol	Proprietary	WiFi / BLE
Internet Access	 No	 Yes
Range	~100m	~50–100m
Power	Very Low	Moderate
Cost	~\$1	~\$2–\$5
Complexity	Low	Medium–High

Choose NRF24L01+ when: simple MCU-to-MCU, low power, no internet needed.

Power Consumption

Mode	Current Draw
Power Down	900 nA
Standby	26 μ A
Receiving (RX)	12–14 mA
Transmitting (TX)	7–11 mA

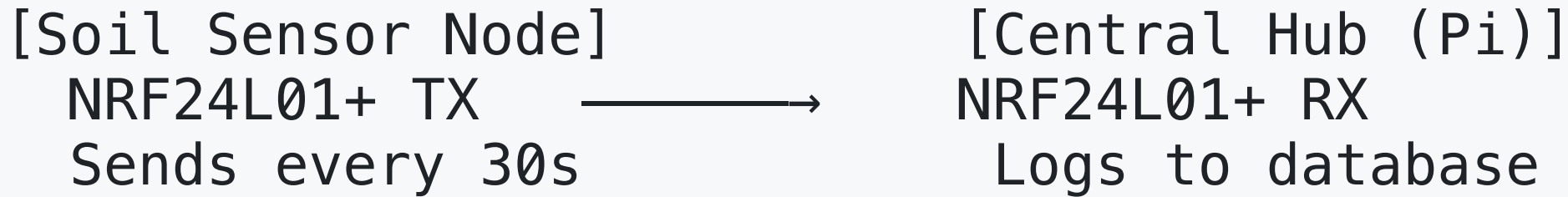
- Very low idle power → great for **battery-powered IoT**
- Wake up, transmit, sleep → saves energy

Common Pitfalls

Problem	Cause	Fix
No communication	Mismatched address	Use exactly the same address
Module not detected	5V power used	Use 3.3V only
Unstable connection	Power supply noise	Add 10–100μF capacitor on VCC
Short range	PA level too low	Increase with <code>RF24_PA_HIGH</code>
Channel interference	WiFi overlap	Switch to channel 76–100

Real-World IoT Example

Scenario: Sensor network in a greenhouse



Why NRF24L01+ here?

- No WiFi needed
- Very low power
- Multiple sensors → multiple pipes (up to 6)
- No pairing required